EXPOSICIÓN DEL PROYECTO

FerreStock

Sistema de inventario y ventas para ferreterías: todo lo que construimos, cómo funciona por dentro y cómo lo publicamos en internet.

Versión 1.9 · 13 de octubre de 2026

Autor: Alan Leiva

Sitio en línea: https://ferrestock-pgyocehooa-nn.a.run.app

CONTENIDO

Qué vamos a ver

1. **¿Qué es FerreStock?** · el problema, la solución y los usuarios
2. **Cómo funciona por dentro** · las tres capas y la nube
3. **Lo que puede hacer** · pantallas y permisos por rol
4. **Lo que hicimos, versión por versión** · del 1.5 al 1.9, con el antes y el después
5. **Tecnologías usadas** · qué es cada una y para qué sirve
6. **Seguridad** · cómo se protegen los datos
7. **Una venta de principio a fin** · el recorrido completo
8. **Publicación en internet** · Google Cloud y actualización automática
9. **Cómo comprobamos que funciona** · pruebas realizadas
10. **Herramientas, retos y conclusiones** · aprendizajes y trabajo futuro

En una frase: FerreStock es una aplicación web, con su propio servidor y base de datos en la nube, que permite a una ferretería llevar su inventario, cobrar con factura, recibir avisos cuando algo se acaba y dar a cada persona del equipo solo lo que le corresponde ver y hacer.

CAPÍTULO 1

¿Qué es FerreStock?

Una ferretería maneja cientos de productos y decenas de entradas y salidas al día. Con cuadernos u hojas de cálculo es fácil perder el control: no se sabe cuánto queda, se descubre tarde que algo se agotó y cualquiera ve costos y precios. **FerreStock** reemplaza eso por un sistema pensado para el mostrador y la bodega.

Antes	Con FerreStock
No se sabe cuánto queda de cada producto	Stock actualizado al instante, ordenado por categoría
Uno se entera tarde de que algo se agotó	Alertas de stock mínimo, notificaciones en la campanita y avisos al vender
Las facturas se hacen a mano	Se cobra, se da el cambio y se imprime la factura en pocos clics
Cualquiera ve costos y precios	Cada persona ve solo lo que le corresponde según su rol
Los datos están en una sola computadora	Los datos están en la nube y el sitio se abre desde cualquier dispositivo

3
roles de usuario

8
tablas en la base de datos

30+
rutas en el servidor

5
versiones documentadas (1.5 a 1.9)

¿Quién lo usa?

Rol	Qué hace en el sistema
Administrador / Administradora	Hace todo: productos, categorías, entradas y salidas, ventas, reportes, usuarios y facturas.
Vendedor / Vendedora	Arma la venta, cobra e imprime la factura. Consulta el inventario en solo lectura (sin costos) y recibe avisos de poco stock y agotados.
Trabajador / Trabajadora de almacén	Consulta cuánto hay de cada producto, sin ver precios ni costos.

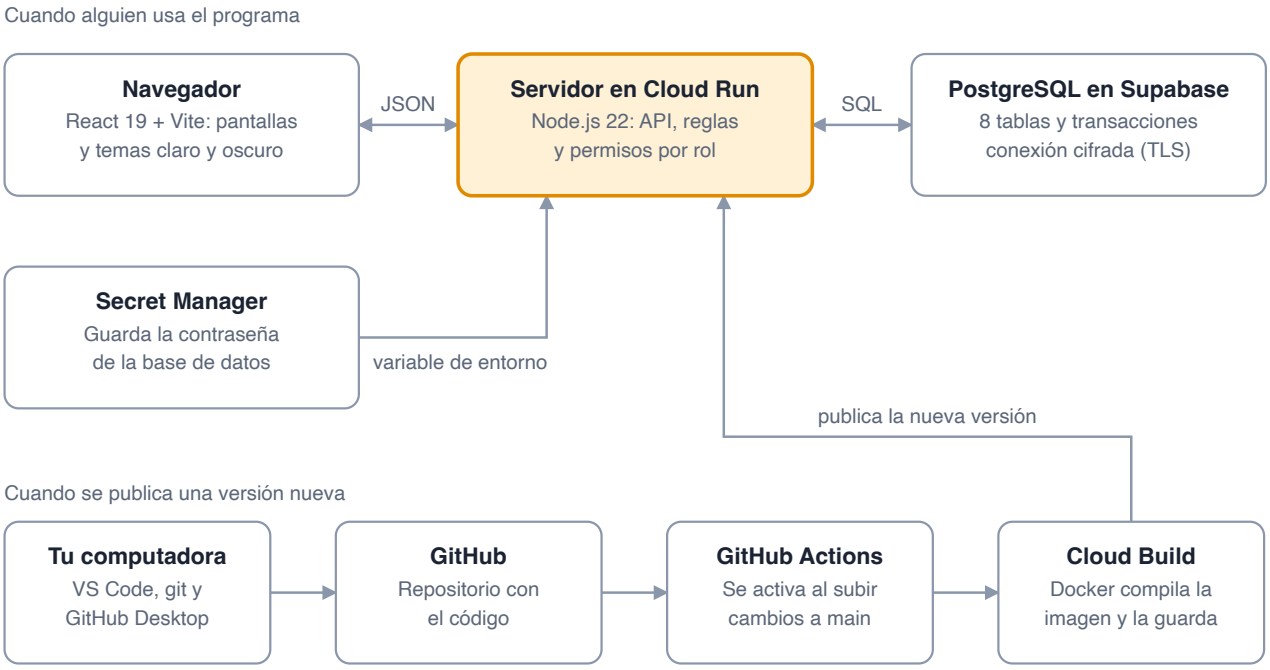
El nombre del rol cambia según el sexo de la persona: una mujer es *Administradora*, *Vendedora* o *Trabajadora de almacén*.

CAPÍTULO 2

Cómo funciona por dentro

FerreStock se divide en **tres capas** que se hablan en cadena. La regla más importante: **la pantalla nunca toca la base de datos**; todo pasa por el servidor, que revisa permisos y reglas. Así, por ejemplo, nadie puede saltarse la regla de no vender más de lo que hay.

FerreStock: tres capas en la nube, publicadas desde GitHub



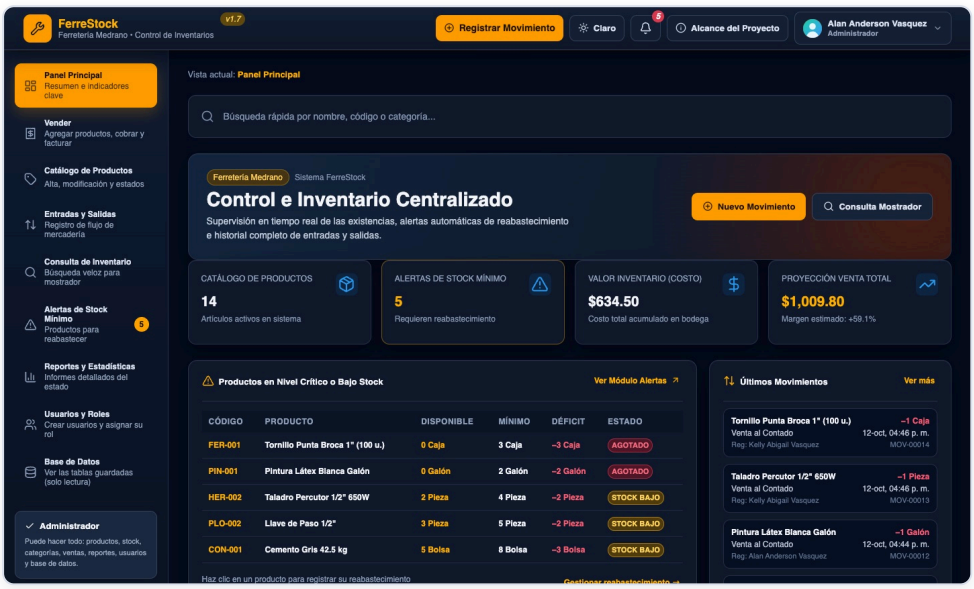
Capa	Qué es	Qué hace
Interfaz	Lo que se ve en el navegador (React)	Muestra pantallas, tablas y formularios, y pide cosas al servidor.
Servidor	Programa en Node.js (en Google Cloud Run)	Recibe los pedidos, valida sesión y rol, aplica las reglas del negocio y consulta la base.
Base de datos	PostgreSQL alojado en Supabase	Guarda productos, ventas, movimientos, usuarios y sesiones, con transacciones.

Debajo del esquema está el camino de una versión nueva: el cambio se sube a GitHub, **GitHub Actions** lo detecta y **Cloud Build** compila y publica, sin que nadie suba nada a mano (capítulo 8).

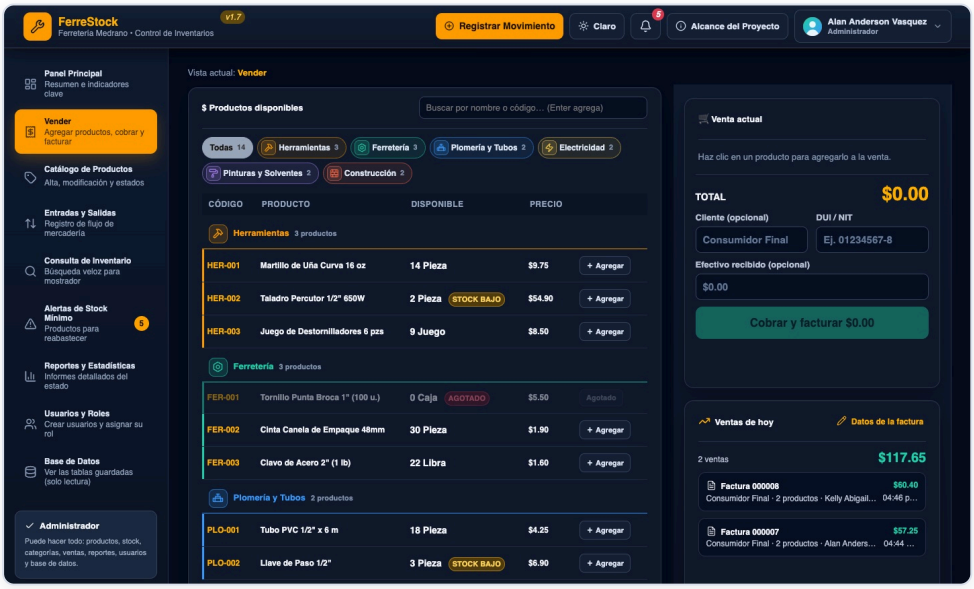
CAPÍTULO 3

Lo que puede hacer

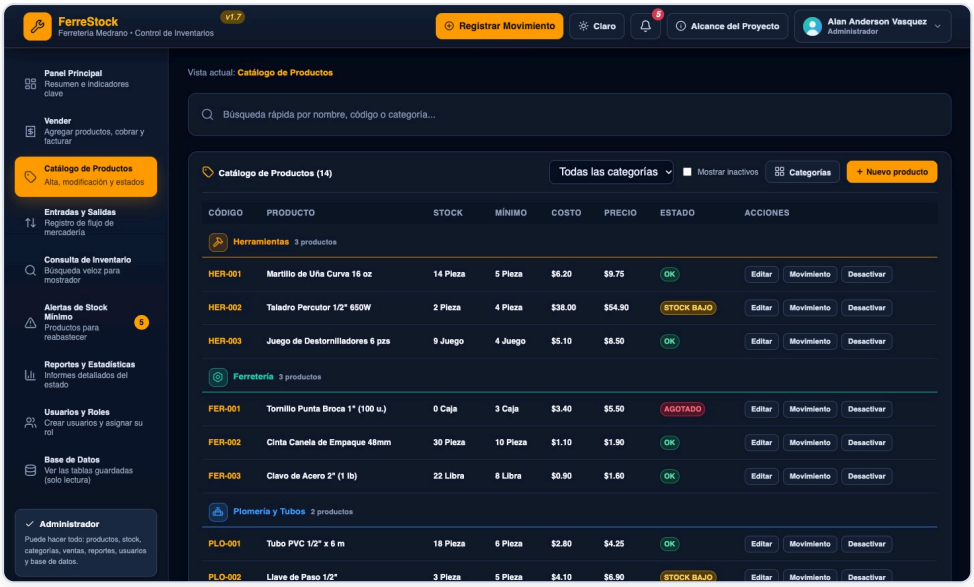
Estas son las pantallas principales del programa (con datos de ejemplo). Cada rol ve solo las que le corresponden.



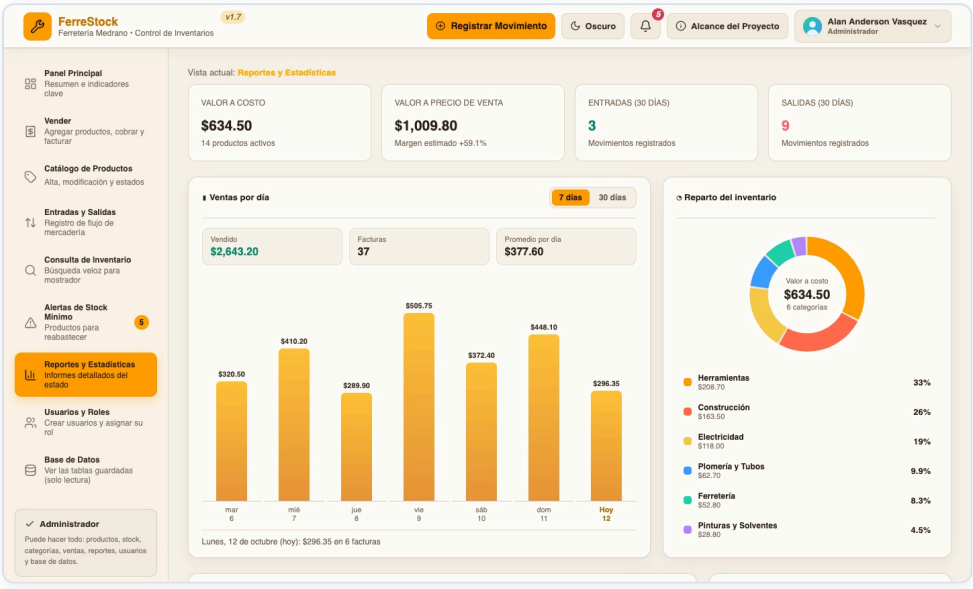
Panel principal: resumen e indicadores clave.



Vender: la caja, con total, cambio y factura.



Catálogo: productos por categoría, con ícono y color.



Reportes: ventas por día y reparto del inventario.

Permisos por rol

Capacidad	Trabajador	Vendedor	Administrador
Ver existencias de cada producto	Sí	Sí	Sí
Ver el precio de venta	No	Sí	Sí
Ver el costo de los productos	No	No	Sí
Cobrar e imprimir facturas	No	Sí	Sí
Consultar inventario y alertas de poco stock	Sí (Stock del almacén)	Sí (solo lectura)	Sí
Crear o editar productos, categorías y movimientos	No	No	Sí
Reportes e historial	No	No	Sí

Capacidad	Trabajador	Vendedor	Administrador
Crear usuarios, asignar roles, fotos y contraseñas	No	No	Sí
Cambiar su propia foto, tema y contraseña	Sí	Sí	Sí

Importante: los permisos se validan **en el servidor** en cada petición, no solo ocultando botones. Un vendedor que intenta pedir la lista de usuarios o crear un producto recibe un error 403 (prohibido). Lo comprobamos con una sesión real.

CAPÍTULO 4

Lo que hicimos, versión por versión

Partimos de la versión 1.5, que ya funcionaba con roles, ventas e inventario, y la fuimos mejorando paso a paso. Cada cambio se probó y se documentó con capturas del antes y el después.

1.5

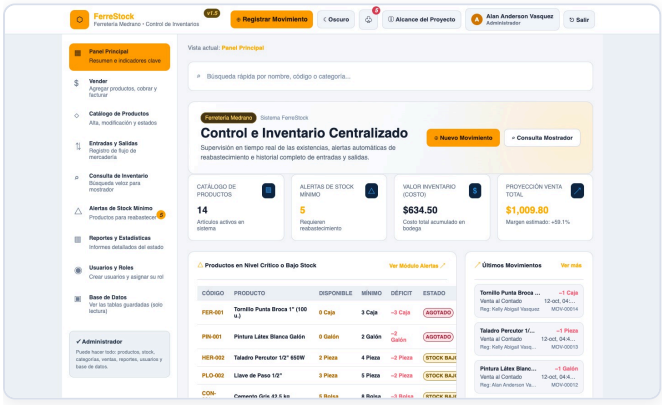
Versión base

Sistema de inventario y ventas con React, Vite y Supabase: tres roles, ventas con factura imprimible, inventario por categorías, entradas y salidas, alertas de stock mínimo, reportes y administración de usuarios. Antes de empezar, ordenamos el código del servidor en carpetas por tema y eliminamos código sin uso.

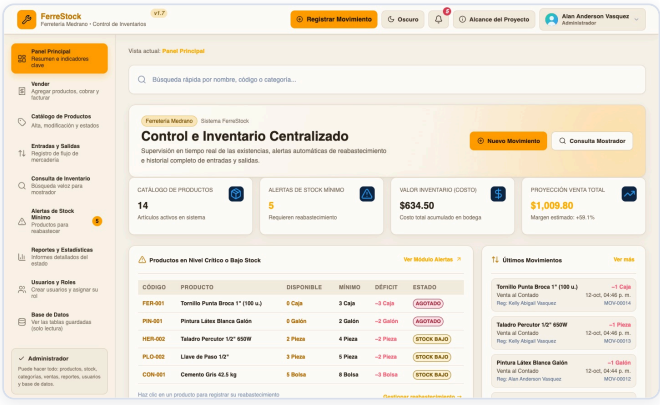
1.6

Nueva cara y cuentas personales

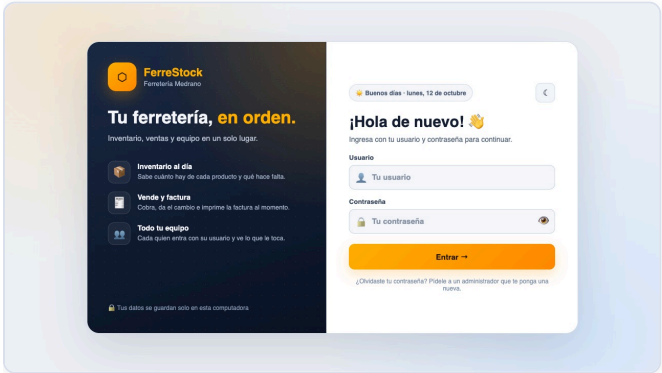
- **Menú de usuario** con foto de perfil, ajustes de cuenta y cierre de sesión en un solo lugar.
- **Ajustes personales** guardados en la nube: tema claro u oscuro, menú compacto, pantalla de inicio y cambio de la propia contraseña.
- **Íconos vectoriales propios** con temática de ferretería, en lugar de emojis.
- **Dos temas** con personalidad propia y una **pantalla de ingreso** distinta en modo claro.
- **Diseño fluido**: ocupa todo el ancho de la pantalla, con menú lateral fijo.



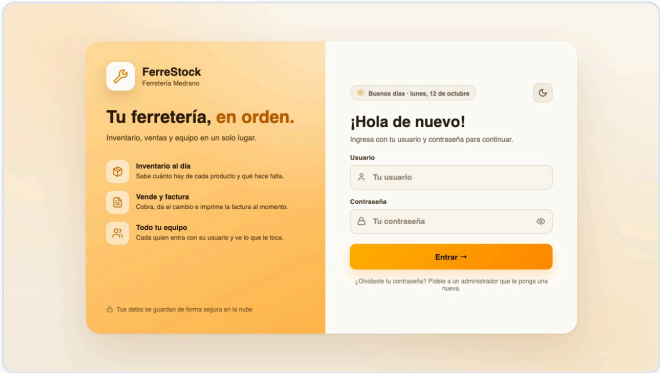
Antes (1.5): ancho fijo, símbolos de texto como íconos.



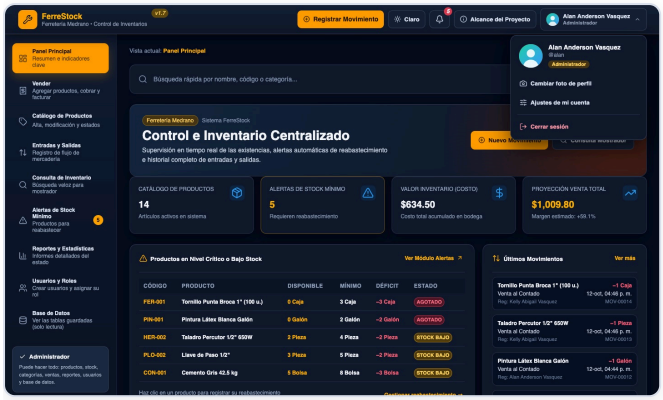
Después (1.6): ancho completo, íconos vectoriales, nuevo tema claro.



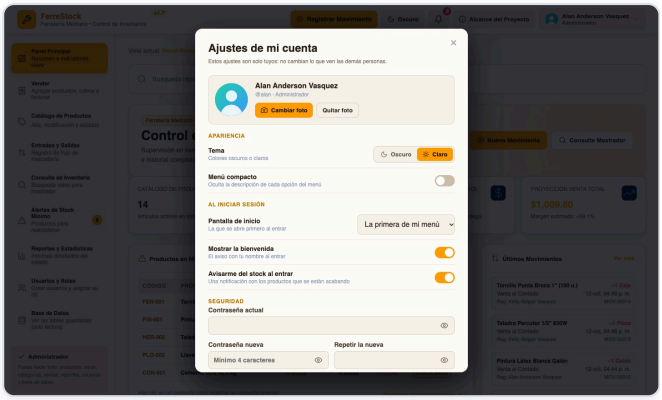
Antes (1.5): ingreso en modo claro.



Después (1.6): ingreso con su propio diseño.



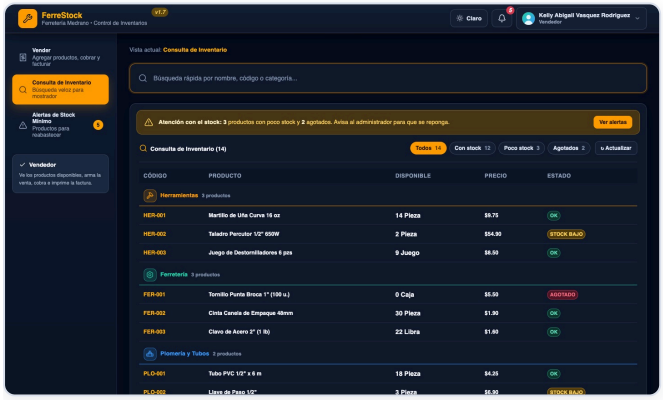
Menú de usuario: foto, ajustes y cerrar sesión.



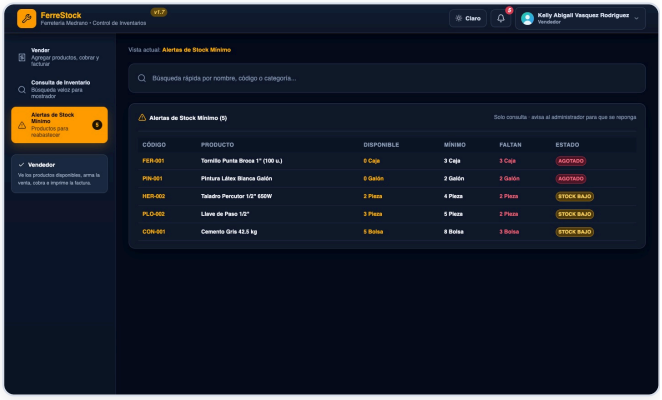
Ajustes de mi cuenta: personalización por usuario.

1.7 El vendedor ve el inventario

- **Consulta de inventario en solo lectura** para el vendedor: sin costos y sin poder modificar nada.
- **Alertas de stock** también para el vendedor, con globo de aviso en el menú y etiquetas de estado al vender.



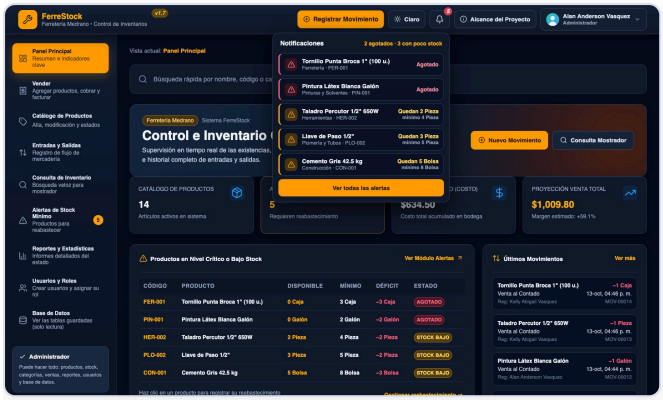
Consulta del vendedor, con el aviso de productos que se acaban.



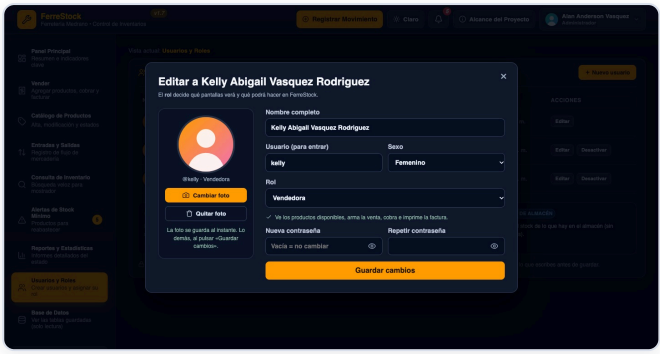
Alertas del vendedor: mismo listado, sin costos ni reabastecer.

1.8 Notificaciones, fotos, sexo y la nube

- **Campanita de notificaciones:** un panel desplegable con los productos agotados o con poco stock, los más urgentes primero.
- **Fotos de perfil gestionadas por el administrador**, desde un panel al costado de la ventana Editar.
- **Sexo de cada persona** y roles en femenino o masculino.
- **Publicación en Google Cloud Run** con actualización automática desde GitHub.



Campanita de notificaciones.

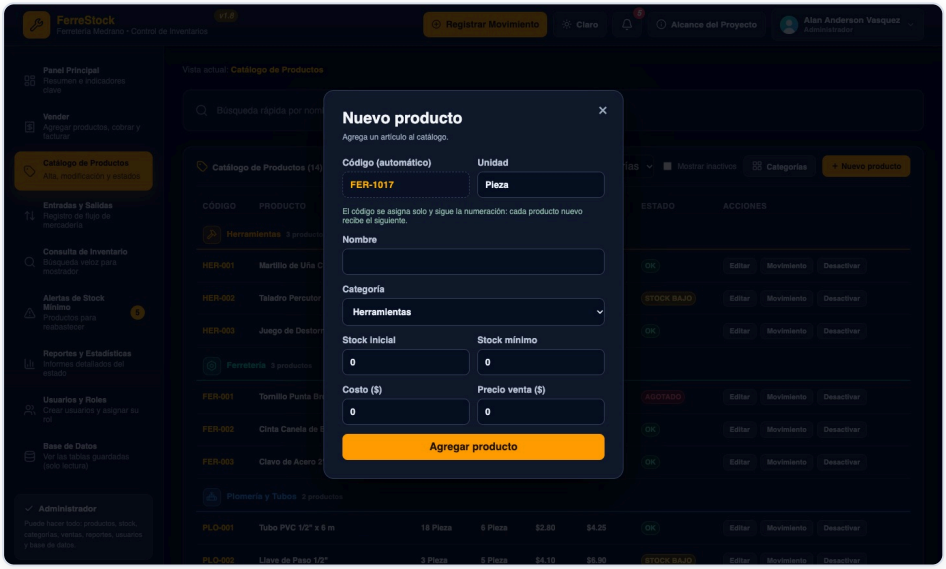


Editar usuario: foto al costado y campo Sexo.

1.9

Código automático de productos

Al crear un producto, el servidor le asigna el siguiente código en orden (FER-1017, FER-1018…). Nadie puede repetir ni saltarse un número, aunque dos personas creen productos a la vez.



Nuevo producto: el código se muestra, pero no se escribe.

+

Ícono del sitio

El sitio tiene su propio ícono (la llave inglesa sobre naranja) en la pestaña del navegador, en favoritos y como aplicación instalable en el celular.



El ícono en pestañas oscura y clara, y en el celular.

CAPÍTULO 5

Tecnologías usadas

FerreStock usa tecnologías gratuitas y estándar de la industria. Casi todo lo visual y todo el servidor es **código propio**: hay muy pocas librerías externas.

La interfaz (lo que ve el usuario)

Tecnología	Qué es	Para qué se usó
JavaScript	El lenguaje de programación	Todo el programa: pantallas y servidor.
React 19	Biblioteca para construir pantallas con piezas reutilizables (componentes)	Todas las pantallas, ventanas y botones.
JSX	HTML escrito dentro del JavaScript	Describir cada pantalla (archivos `.jsx`).
Vite 8	Herramienta de desarrollo y compilación	Abrir el programa mientras se desarrolla y compilarlo para publicar.
CSS propio	Lenguaje de estilos	Colores, temas claro y oscuro, diseño que se adapta a la pantalla, animaciones.
SVG	Imágenes vectoriales	Íconos propios que se ven nítidos a cualquier tamaño.
Funciones del navegador	`fetch`, `localStorage`, `canvas`, impresión	Hablar con el servidor, recordar el tema, reducir la foto de perfil e imprimir facturas.

El servidor y la base de datos

Tecnología	Qué es	Para qué se usó
Node.js 22	Ejecuta JavaScript fuera del navegador	El servidor, con servidor HTTP propio y sin frameworks.
API en JSON	Conjunto de rutas (/api/…) que atiende el servidor	Más de 30 rutas, cada una con los roles que pueden usarla.
`pg` **8**	Conector de Node a PostgreSQL	Única librería externa del servidor.
PostgreSQL	Base de datos relacional de código abierto	Guardar toda la información en 8 tablas.
Supabase	Servicio que aloja PostgreSQL en la nube	Tener la base de datos disponible desde cualquier lugar.
SQL	Lenguaje de consultas	Leer y guardar datos, con parámetros para evitar inyección SQL.

Tecnología	Qué es	Para qué se usó
Transacciones y bloqueos	Cambios que se guardan completos o no se guardan; candados entre peticiones	Evitar ventas a medias, ventas dobles y códigos repetidos.

> Las ocho tablas son: `categorias`, `productos`, `movimientos`, `ventas`, `usuarios`, `roles`, `sesiones` y `negocio`.

Publicación y nube

Tecnología	Qué es	Para qué se usó
Docker	Empaquetador de programas	Meter el programa y todo lo que necesita en una imagen que corre igual en cualquier lugar.
Google Cloud Run	Servicio que ejecuta la imagen	Dar el link público con HTTPS.
Cloud Build / Artifact Registry	Compilador y almacén de imágenes de Google	Construir y guardar cada versión publicada.
Secret Manager	Caja fuerte de secretos	Guardar la contraseña de la base fuera del código.
Git y GitHub	Historial de cambios y su alojamiento	Guardar el código y activar la publicación automática.
GitHub Actions	Tareas automáticas de GitHub	Publicar solo al subir cambios.
Workload Identity Federation	Identificación entre servicios sin claves	Que GitHub se conecte a Google sin guardar ninguna clave.

CAPÍTULO 6

Seguridad

La seguridad se resuelve en el servidor, porque todo lo que ocurre en el navegador puede ser manipulado por quien lo use.

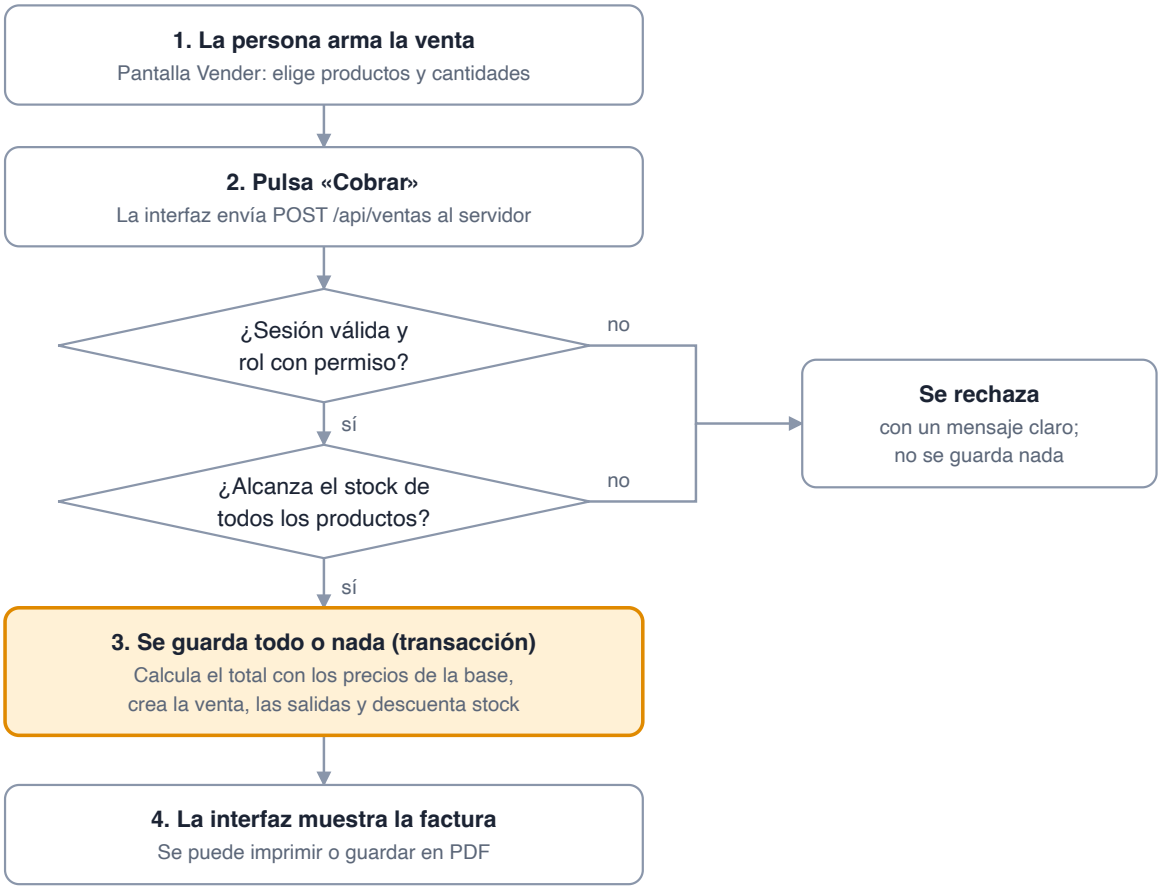
Medida	Cómo funciona
Contraseñas cifradas	Se guardan con **scrypt** y una sal distinta por persona. No se pueden leer ni recuperar, solo comprobar.
Sesiones	Una llave aleatoria viaja en una cookie `HttpOnly`, `SameSite=Strict` y `Secure` en la nube. En la base solo se guarda su huella. Duran 12 horas.
Roles y permisos	Se validan en el servidor en cada petición. El vendedor no recibe costos; el trabajador no recibe costos ni precios.
Intentos fallidos	Cinco contraseñas incorrectas seguidas bloquean al usuario un minuto.
Conexión cifrada	TLS con verificación del certificado de Supabase.
Validación de datos	Las fotos solo pueden ser JPG, PNG o WebP de hasta 90 KB (no SVG); los ajustes se comparan con una lista permitida; cada petición tiene tamaño máximo.
Protecciones del navegador	En producción se envían `Content-Security-Policy`, `X-Frame-Options`, `nosniff` y `Referrer-Policy`; solo se entrega la carpeta de pantallas compiladas.
Secretos fuera del código	La contraseña de la base está en `.env.local` (ignorado por git) y, en la nube, en Secret Manager.
Permisos mínimos en la nube	La cuenta que publica solo puede publicar este servicio, y solo acepta peticiones de este repositorio y esta rama.

CAPÍTULO 7

Una venta de principio a fin

Cobrar una venta muestra cómo trabajan juntas todas las piezas: la interfaz pide, el servidor revisa y decide, y la base de datos guarda.

Una venta se guarda completa o no se guarda



Si la sesión no es válida o no alcanza el stock, el servidor rechaza la venta y la base de datos **no cambia**: por eso nunca queda una venta a medias. Los precios se leen siempre de la base, no de lo que muestra la pantalla.

CAPÍTULO 8

Publicación en internet

FerreStock está publicado en **Google Cloud Run** y se abre desde cualquier computadora o celular con internet:

> **https://ferrestock-pgyocehooa-nn.a.run.app**

Cómo se publica

1. Se cambia el código en la computadora (Visual Studio Code) y se prueba.
2. Se sube a GitHub con **Push origin** (GitHub Desktop).
3. **GitHub Actions** detecta el cambio en la rama `main`.
4. Se identifica ante Google **sin guardar claves** (Workload Identity Federation).
5. **Cloud Build** compila la imagen con Docker y **Cloud Run** la publica.
6. En unos minutos el link muestra la versión nueva. Si algo falla, la versión anterior sigue funcionando.

Qué conviene saber

- Los cambios que solo tocan documentación (`.md` o la carpeta `docs/`) **no** republican el sitio.
- El sitio se enciende cuando hay visitas: la primera tras un rato sin uso tarda unos segundos más.
- Se limitó el servicio a 2 instancias de 512 MB, en la región de Montreal, la más cercana a la base de datos.
- Se puede instalar como aplicación desde el navegador (manifiesto web y ícono propios).
- Para compartirlo se creó un usuario de rol vendedor; no hace falta entregar la cuenta del administrador.

CAPÍTULO 9

Cómo comprobamos que funciona

Cada cambio se probó antes de darlo por bueno, con pruebas hechas a la medida:

Qué se comprobó	Cómo
Reglas del servidor (roles, fotos, sexo, códigos)	Pruebas contra la base real dentro de una **transacción que se revierte** al final: se prueba de verdad sin dejar ningún dato.
Códigos automáticos sin repetir	Cuatro peticiones simultáneas con transacciones independientes dieron FER-1017 a FER-1020, distintos y consecutivos.
Permisos	Con una sesión real de vendedor, las rutas de administrador respondieron 403 (prohibido).
Seguridad del servidor de producción	Se intentó leer archivos de configuración por varias rutas: siempre se entregó la pantalla de la aplicación o un 403, nunca el archivo.
Pantallas y flujos	Con un navegador real automatizado y datos de ejemplo: abrir ventanas, subir fotos, quitar y confirmar, cambiar de tema y de tamaño de pantalla.
Publicación en la nube	La publicación automática se ejecutó con éxito y se verificó desde fuera: pantalla, API, cookie segura y encabezados de seguridad.
Íconos e instalación	Chrome validó el manifiesto sin errores y sin problemas de instalación.

> **Pendiente:** todavía no hay pruebas automáticas que se ejecuten solas con cada cambio. Es el siguiente paso natural del proyecto.

CAPÍTULO 10

Herramientas, retos y conclusiones

Herramientas de desarrollo

- **Visual Studio Code** (editor), **Node.js y npm**, **Git y GitHub Desktop**, la terminal y la herramienta **gcloud** de Google.
- **Google Chrome** para ver el programa y tomar las capturas.
- **Markdown** para la documentación (`README.md` y `docs/`).
- **Claude Code**, un asistente de inteligencia artificial de Anthropic, que ayudó a escribir, probar y documentar el código siguiendo las indicaciones del desarrollador. Se declara aquí por transparencia.

Retos y cómo se resolvieron

Reto	Solución
Dos personas podían recibir el mismo código de producto	El servidor asigna el código y las escrituras se ordenan con un candado de la base de datos.
La contraseña de la base iba entre comillas en un archivo y la publicación fallaba	El script de publicación aprendió a quitar las comillas igual que el programa.
Un proyecto nuevo de Google no dejaba compilar	Se dio a la cuenta de compilación el permiso mínimo que Google recomienda.
La herramienta de Google necesitaba una versión de Python más reciente que la del equipo	Se instaló un Python aparte, solo para esa herramienta.
Los datos de ejemplo del documento no debían ser reales	Se montó una API de mentira con datos ficticios para tomar todas las capturas.

Conclusiones

FerreStock pasó de ser un programa que solo funcionaba en una computadora a un **sistema publicado en internet**, con cuentas personales, permisos por rol, avisos de stock, códigos automáticos y actualización automática. Se mantuvo el código ordenado por carpetas y documentado en `docs/DOCUMENTACION.md`, y cada versión quedó registrada con capturas.

Mejoras futuras

- Pruebas automáticas que se ejecuten con cada cambio.
- Rotar periódicamente la contraseña de la base de datos y respaldarla de forma automática.
- Facturación electrónica certificada (DTE) ante Hacienda: hoy la factura es un comprobante interno.
- Notificaciones en tiempo real entre computadoras, sin recargar.
- Exportar reportes a PDF y Excel, y un dominio propio para el sitio.